DESENVOLVIMENTO DE APLICATIVOS MOBILE



MÓDULO 07

Persistência Local

Opções de armazenamento, DataStore, Room, relações, migrações e integração com a arquitetura

Prof. Dr. Francisco
IFMA – Campus São Luís – Monte Castelo
Departamento de Eletroeletrônica (DEE)

1 de novembro de 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Escolher a forma de armazenamento adequada a cada tipo de dado.
- Guardar preferências do usuário com **DataStore**.
- Modelar e consultar um banco SQLite com **Room**: entidades, DAOs e banco.
- Representar relações entre tabelas e converter tipos.
- Evoluir o esquema do banco com migrações sem perder dados.
- Integrar o banco ao repositório e expor dados reativos com `Flow`.

Roteiro do módulo

1. Opções de armazenamento
2. DataStore
3. Room
4. Integração com a arquitetura
5. Fechamento

1
Opções de armazenamento

Onde guardar cada coisa

Mecanismo	Indicado para	Exemplo no LabMonitor
DataStore Preferences	pares chave-valor pequenos	tema, unidade (°C/°F), limite de alerta
Proto DataStore	objeto de configuração tipado	perfil do usuário em cache
Room (SQLite)	dados estruturados, consultas, relações	equipamentos, bancadas, leituras
Arquivos internos	arquivos privados do app	CSV exportado, logs
Cache (`cacheDir`)	dados descartáveis	miniaturas baixadas
MediaStore / **seletor**	arquivos compartilhados com o usuário	fotos de equipamentos na galeria

Armazenamento interno

Pasta privada do app (`filesDir`). Outros apps não acessam, e ela é apagada na desinstalação.

SharedPreferences

A API antiga é síncrona e pode travar a UI. Em código novo, use DataStore.

2
DataStore

DataStore Preferences

AjustesRepository.kt

```
private val Context.ajustes by preferencesDataStore(
  name = "ajustes")

class AjustesRepository @Inject constructor(
  @ApplicationContext private val ctx: Context
) {
  private object Chaves {
    val LIMITE = doublePreferencesKey("limite_alerta")
    val FAHRENHEIT = booleanPreferencesKey("usar_f")
  }

  val limite: Flow<Double> = ctx.ajustes.data
    .catch { emit(emptyPreferences()) }  // erro de I/O
    .map { it[Chaves.LIMITE] ?: 30.0 }

  suspend fun definirLimite(valor: Double) {
    ctx.ajustes.edit { it[Chaves.LIMITE] = valor }
  }
}
```

Características

- Leitura como `Flow`: a UI atualiza sozinha.
- Escrita transacional com `edit`.
- Main-safe: I/O fora da thread principal.

Dependência

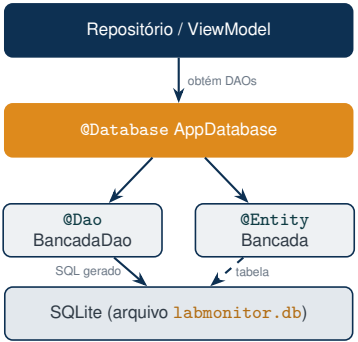
`androidx.datastore:datastore-preferences`

Atenção

Uma única instância por arquivo (o *delegate* garante). Listas grandes vão para o Room.

3
Room

Arquitetura do Room



Room é uma camada sobre o SQLite que:

- verifica o SQL **em tempo de compilação**;
- converte linhas em objetos Kotlin automaticamente;
- devolve `Flow` que reemite quando a tabela muda;
- suporta coroutines e migrações;
- funciona também em Kotlin Multiplatform.

Dependências

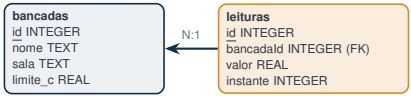
`room-runtime`, `room-ktx` e `ksp(room-compiler)`.

Entidades

Entidades.kt

```kotlin
@Entity(tableName = "bancadas")
data class BancadaEntity(
  @PrimaryKey val id: Int,
  val nome: String,
  val sala: String,
  @ColumnInfo(name = "limite_c") val limite: Double = 30.0
)

@Entity(
  tableName = "leituras",
  foreignKeys = [ForeignKey(
    entity = BancadaEntity::class,
    parentColumns = ["id"], childColumns = ["bancadaId"],
    onDelete = ForeignKey.CASCADE)],
  indices = [Index("bancadaId")]
)
data class LeituraEntity(
  @PrimaryKey(autoGenerate = true) val id: Long = 0,
  val bancadaId: Int,
  val valor: Double,
  val instante: Instant
)
```



Anotações

`@Entity` vira tabela, `@PrimaryKey` define a chave e `@ColumnInfo` renomeia colunas. `ForeignKey` garante a integridade e o `CASCADE` apaga as leituras junto com a bancada.

Dica

Crie `Index` nas colunas usadas em `WHERE` e em chaves estrangeiras.

DAO: *Data Access Object*

BancadaDao.kt

```
@Dao
interface BancadaDao {
  @Query("SELECT * FROM bancadas ORDER BY nome")
  fun observarTodas(): Flow<List<BancadaEntity>>

  @Query("SELECT * FROM bancadas WHERE id = :id")
  suspend fun buscar(id: Int): BancadaEntity?

  @Upsert                          // insere ou atualiza
  suspend fun salvar(vararg b: BancadaEntity)

  @Delete suspend fun remover(b: BancadaEntity)
  @Query("""SELECT AVG(valor) FROM leituras
    WHERE bancadaId = :id AND instante >= :desde""")
  fun media(id: Int, desde: Instant): Flow<Double?>
  @Transaction
  suspend fun substituirTodas(lista: List<BancadaEntity>) {
    apagarTodas(); salvar(*lista.toTypedArray())
  }
  @Query("DELETE FROM bancadas") suspend fun apagarTodas()
}
```

Retorno	Comportamento
`Flow<T>`	observa: reemite a cada mudança
`suspend fun`	executa uma vez, fora da UI

Parâmetros

`:id` na consulta liga-se ao parâmetro `id` da função. O Room valida nomes de tabelas e colunas na compilação.

`@Transaction`

Agrupa operações: ou todas acontecem, ou nenhuma. Evita que a UI veja a tabela vazia no meio da troca.

Banco de dados e conversores de tipo

AppDatabase.kt

```
@Database(
  entities = [BancadaEntity::class,
              LeituraEntity::class],
  version = 1,
  exportSchema = true
)
@TypeConverters(Conversores::class)
abstract class AppDatabase : RoomDatabase() {
  abstract fun bancadaDao(): BancadaDao
  abstract fun leituraDao(): LeituraDao
}

class Conversores {
  @TypeConverter
  fun deInstant(i: Instant?): Long? = i?.toEpochMilli()
  @TypeConverter
  fun paraInstant(ms: Long?): Instant? =
    ms?.let(Instant::ofEpochMilli)
}
```

Conversores

O SQLite só conhece inteiro, real, texto e blob. `@TypeConverter` ensina o Room a guardar `Instant`, `enum` e outros tipos.

Criação via Hilt (Módulo 06)

`Room.databaseBuilder(...)` em um `@Provides @Singleton`: uma instância única para o app inteiro.

Dica

`createFromAsset("inicial.db")` cria o banco já preenchido (ex.: inventário do laboratório).

Relações entre tabelas

Um-para-muitos

```
data class BancadaComEquipamentos(
  @Embedded val bancada: BancadaEntity,
  @Relation(
    parentColumn = "id",
    entityColumn = "bancadaId"
  )
  val equipamentos: List<EquipamentoEntity>
)

@Dao
interface BancadaDao {
  @Transaction
  @Query("SELECT * FROM bancadas WHERE id = :id")
  fun comEquipamentos(id: Int):
    Flow<BancadaComEquipamentos?>
}
```

`@Embedded` + `@Relation`

O Room faz duas consultas e monta o objeto composto: a bancada e a lista dos seus equipamentos.

Muitos-para-muitos

Crie uma tabela de junção (ex.: `ReservaUsuario`) e use `@Relation(associateBy = Junction(...))`.

Atenção

Use `@Transaction` em consultas com `@Relation` para que as duas consultas leiam um estado consistente.

Migrações: evoluindo o esquema

De v1 para v3

```
@Database(entities = [...], version = 3,
  autoMigrations = [AutoMigration(from = 1, to = 2)])
abstract class AppDatabase : RoomDatabase()

// mudanças complexas: migração manual
val MIGRACAO_2_3 = object : Migration(2, 3) {
  override fun migrate(db: SupportSQLiteDatabase) {
    db.execSQL("ALTER TABLE bancadas " +
      "ADD COLUMN responsavel TEXT")
    db.execSQL("CREATE INDEX idx_leitura_inst " +
      "ON leituras(instante)")
  }
}

Room.databaseBuilder(ctx, AppDatabase::class.java,
      "lab.db")
  .addMigrations(MIGRACAO_2_3)
  .build()
```

Regra

Mudou uma entidade? **Incremente a versão** e forneça a migração, senão o app trava ao abrir o banco antigo.

`exportSchema`

Gera um JSON do esquema de cada versão. Versione esses arquivos: são a base das *auto migrations* e dos testes de migração.

Erro comum

`fallbackToDestructiveMigration()` apaga os dados do usuário. Aceitável só em protótipos ou caches.

4
Integração com a arquitetura

Fluxo reativo de ponta a ponta



A mágica

Depois que o DAO grava, o Room **notifica** todos os `Flow` que leem aquela tabela. A tela se atualiza sem nenhum código extra de “recarregar”.

Inspecionando

App Inspection → **Database Inspector** mostra as tabelas do app em execução, permite rodar SQL e atualiza ao vivo.

Guardando leituras de sensores

LeituraRepository.kt

```
class LeituraRepository @Inject constructor(
  private val dao: LeituraDao
) {
  fun ultimas(bancada: Int, n: Int = 120) =
    dao.ultimas(bancada, n)          // Flow

  suspend fun registrar(l: Leitura) =
    dao.inserir(l.paraEntidade())

  // mantém o banco pequeno: apaga o que é antigo
  suspend fun limpar(dias: Long = 30) =
    dao.apagarAntesDe(
      Instant.now().minus(dias, ChronoUnit.DAYS))
}

@Query("""SELECT * FROM leituras WHERE bancadaId = :b
  ORDER BY instante DESC LIMIT :n""")
fun ultimas(b: Int, n: Int): Flow<List<LeituraEntity>>
```

Volume de dados

Um sensor a 1 Hz gera 86 400 linhas por dia. Planeje:

- retenção (apagar o que é antigo);
- agregação (médias por minuto);
- consultas com `LIMIT` e índices.

Dica

Para inserir muitas leituras, agrupe em lote (`@Insert` com `List`) em vez de uma transação por leitura.

Boas práticas e segurança

Boas práticas

- Nunca acesse o banco na thread principal (o Room bloqueia por padrão).
- Uma instância de `AppDatabase` por processo.
- Teste as migrações com `MigrationTestHelper`.
- Banco em memória (`inMemoryDatabaseBuilder`) nos testes (Módulo 11).

Dados sensíveis

- Não guarde senhas. Guarde *tokens* e prefira o **Android Keystore** para chaves criptográficas.
- O backup automático do Android copia arquivos do app para a nuvem do usuário; exclua o que não deve sair do aparelho com `dataExtractionRules`.
- Trate dados pessoais conforme a **LGPD** (Módulo 12).

5
Fechamento

Resumo

Resumo

- Preferências pequenas no DataStore; dados estruturados no Room; arquivos em `filesDir`.
- Room = `@Entity` (tabelas) + `@Dao` (consultas) + `@Database` (o banco).
- DAOs devolvem `Flow` para observar e usam `suspend` para operações pontuais.
- `@TypeConverter` para tipos especiais; `@Relation` para relações.
- Toda mudança de esquema exige nova versão e migração.
- Com repositório + `Flow`, gravar no banco atualiza a tela automaticamente.

Exercícios

Exercício 1 — DataStore

Crie a tela de Ajustes com limite de alerta (`Slider`), unidade de temperatura (`Switch`) e intervalo de amostragem. Todos os valores devem persistir.

Exercício 2 — Consultas

Escreva consultas DAO para: as 5 bancadas com maior média de temperatura nas últimas 24 h; a contagem de leituras acima do limite por bancada; leituras entre duas datas.

Exercício 3 — Migração

Adicione a coluna `responsavel` à tabela de bancadas e escreva a migração 1→2. Confirme no Database Inspector que os dados antigos foram preservados.

LabMonitor — Etapa 7

Substitua os repositórios *fake* por Room: bancadas, equipamentos e leituras do simulador. Guarde o limite de alerta no DataStore e implemente a limpeza de leituras com mais de 30 dias.

Para aprofundar

- Visão geral de armazenamento: `https://developer.android.com/training/data-storage`
- DataStore: `https://developer.android.com/topic/libraries/architecture/datastore`
- Room: `https://developer.android.com/training/data-storage/room`
- Relações no Room:
  `https://developer.android.com/training/data-storage/room/relationships`
- Migrações:
  `https://developer.android.com/training/data-storage/room/migrating-db-versions`
- Backup automático: `https://developer.android.com/identity/data/autobackup`

Fim do Módulo 07

Próximo: Módulo 08 — Rede e consumo de APIs: HTTP, REST, Retrofit, serialização, erros e estratégia offline-first.